ACCESO A BASES DE DATOS SQL EN JAVA

JDBC, el patrón DAO y una implementación con MySQL

DESARROLLO DE SISTEMAS ORIENTADO A OBJETOS · TECNICATURA EN DESARROLLO DE SOFTWARE

Prof. Esteban Santini · Septiembre 2026

¿QUÉ ES JDBC?

La conexión estándar entre Java y las bases de datos

JDBC (Java Database Connectivity) es la API estándar de Java para conectarse a bases de datos relacionales y ejecutar sentencias SQL desde código Java.

Cada motor de base de datos (MySQL, PostgreSQL, Oracle, SQL Server) provee su propio driver JDBC: una biblioteca que implementa las interfaces de java.sql (Connection, Statement, ResultSet) para ese motor en particular.

Java programa siempre contra esas mismas interfaces. Es el driver el que traduce cada llamada al protocolo específico de la base de datos, sin que el resto del programa lo note.

TIPOS DE DRIVERS JDBC

TIPO 1 · PUENTE JDBC-ODBC

Traduce las llamadas JDBC al estándar ODBC del sistema operativo. Está en desuso.

TIPO 2 · API NATIVA

Convierte JDBC en llamadas a la API nativa del cliente del motor. También en desuso.

TIPO 3 · PROTOCOLO DE RED

Un servidor intermedio traduce JDBC al protocolo del motor. Poco usado hoy.

TIPO 4 · PROTOCOLO NATIVO

100% Java: habla directo con el motor por su protocolo propio. Es el que usan MySQL y PostgreSQL.

DESCARGAR EL DRIVER JDBC

No viene incluido ni con el JDK ni con NetBeans

1. Ir a dev.mysql.com/downloads/connector/j, la página oficial del conector.

2. En “Select Operating System” elegir “Platform Independent”: el driver es un .jar.

3. Descargar el ZIP: no hace falta crear cuenta, alcanza con “No thanks, just start my download”.

4. Descomprimir el archivo: adentro está el .jar del driver, listo para usar en cualquier proyecto:

```
mysql-connector-j-9.5.0.jar
```

AGREGAR EL DRIVER EN ANT

Se agrega a mano en las Properties del proyecto

1. Clic derecho sobre el proyecto → Properties → categoría Libraries.

2. En la pestaña Compile, clic en “Add JAR/Folder…” y elegir el .jar descargado.

3. NetBeans lo agrega al classpath de compilación y ejecución del proyecto.

4. Para compartir el proyecto conviene copiar el .jar a una carpeta lib/ propia:

```
lib/mysql-connector-j-9.5.0.jar
```

AGREGAR EL DRIVER CON MAVEN

Se descarga automáticamente desde Maven Central

1. Abrir el pom.xml del proyecto (NetBeans lo crea solo en un proyecto Maven).

2. Agregar la dependencia del conector dentro de <dependencies>:

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>9.5.0</version>
</dependency>
```

3. Al guardar, NetBeans descarga el .jar desde Maven Central automáticamente.

FLUJO BÁSICO DE TRABAJO CON JDBC

Los pasos que se repiten en cualquier operación

1. Registrar u obtener el driver del motor de base de datos.

2. Abrir una Connection con DriverManager.getConnection(url, usuario, clave).

3. Crear un Statement o, preferentemente, un PreparedStatement.

4. Ejecutar la sentencia SQL (executeUpdate o executeQuery).

5. Si hay resultados, recorrer el ResultSet fila por fila.

6. Cerrar los recursos, idealmente con try-with-resources.

LA ENTIDAD PERSONA

Clase que representa un registro de la tabla personas

```
public class Persona {
    private Long id;
    private String nombre;
    private String apellido;
    private String documento;

    public Persona(String nombre, String apellido, String documento) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.documento = documento;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNombre() { return nombre; }
    public String getApellido() { return apellido; }
    public String getDocumento() { return documento; }
}
```



Alta de una Persona usando PreparedStatement

```
String url = "jdbc:mysql://localhost:3306/escuela";
String sql = "INSERT INTO personas (nombre, apellido, documento) VALUES (?, ?, ?)";

try (Connection cn = DriverManager.getConnection(url, "root", "clave");
     PreparedStatement ps = cn.prepareStatement(sql)) {

    ps.setString(1, "Juan");
    ps.setString(2, "Pérez");
    ps.setString(3, "30111222");
    ps.executeUpdate();

} catch (SQLException e) {
    e.printStackTrace();
}
```

LISTADO DE PERSONAS CON JDBC

Recorriendo un ResultSet con Statement

```
String sql = "SELECT nombre, apellido, documento FROM personas";

try (Connection cn = DriverManager.getConnection(url, "root", "clave");
     Statement st = cn.createStatement();
     ResultSet rs = st.executeQuery(sql)) {

    while (rs.next()) {
        String nombre = rs.getString("nombre");
        String apellido = rs.getString("apellido");
        System.out.println(apellido + ", " + nombre);
    }

} catch (SQLException e) {
    e.printStackTrace();
}
```

JDBC DIRECTO vs. PATRÓN DAO

JDBC DIRECTO

- **Rápido para prototipos**

Sirve para probar una consulta puntual sin mucha estructura alrededor.

- **SQL mezclado con la lógica**

El código de acceso a datos queda pegado a la lógica de negocio.

- **Difícil de reutilizar**

Hay que repetir la conexión y el manejo de errores en cada método.

PATRÓN DAO

- **Separa responsabilidades**

El acceso a datos queda aislado detrás de una interfaz clara.

- **Reutilizable y testeable**

Las operaciones CRUD se escriben una vez y se usan desde toda la app.

- **Fácil de cambiar de motor**

Cambiar de MySQL a otro motor implica tocar una sola clase.

¿QUÉ ES EL PATRÓN DAO?

Data Access Object

DAO es un patrón de diseño que encapsula el acceso a una fuente de datos (una base de datos, un archivo, una API) detrás de una interfaz.

Separa las operaciones CRUD (crear, leer, actualizar, eliminar) del resto de la aplicación: la lógica de negocio programa contra la interfaz, no contra JDBC directamente.

Esto permite cambiar la implementación (MySQL, PostgreSQL, una versión en memoria para los tests) sin modificar el resto del sistema.

LA EXCEPCIÓN DAOEXCEPTION



Es checked: hay que declararla o capturarla

```
public class DaoException extends Exception {

    public DaoException(String message, Throwable cause) {
        super(message, cause);
    }

    public DaoException(String message) {
        super(message);
    }
}
```

INTERFAZ GENÉRICA DAO<T, ID>

Contrato común para cualquier entidad

```
public interface DAO<T, ID> {

    void insert(T entity) throws DaoException;

    T findById(ID id) throws DaoException;

    List<T> findAll() throws DaoException;

    void update(T entity) throws DaoException;

    void delete(ID id) throws DaoException;
}
```

INTERFAZ PERSONADAO

Especialización de DAO<T, ID> para Persona

```
public interface PersonaDAO extends DAO<Persona, Long> {

    List<Persona> findByLastName(String lastName) throws DaoException;
}
```

IMPLEMENTACIÓN MYSQL: BUSCAR ID



Buscar una Persona por su id

```
@Override
public Persona findById(Long id) throws DaoException {
    String sql = "SELECT id, nombre, apellido, documento FROM personas WHERE id = ?";
    try (Connection cn = DriverManager.getConnection(url, usuario, clave);
         PreparedStatement ps = cn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            Persona p = new Persona(
                rs.getString("nombre"),
                rs.getString("apellido"),
                rs.getString("documento"));
            p.setId(rs.getLong("id"));
            return p;
        }
        return null;
    } catch (SQLException e) {
        throw new DaoException("No se pudo buscar la persona con id " + id, e);
    }
}
```

RECUPERAR EL ID GENERADO

RETURN_GENERATED_KEYS: la clave que generó la base

```
@Override
public void insert(Persona p) throws DaoException {
    String sql = "INSERT INTO personas (nombre, apellido, documento) VALUES (?, ?, ?)";
    // RETURN_GENERATED_KEYS le pide a JDBC que exponga la clave autogenerada
    try (Connection cn = DriverManager.getConnection(url, usuario, clave);
         PreparedStatement ps = cn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, p.getNombre());
        ps.setString(2, p.getApellido());
        ps.setString(3, p.getDocumento());
        ps.executeUpdate();
        // getGeneratedKeys() devuelve un ResultSet posicionado en la clave generada
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) {
                p.setId(keys.getLong(1));
            }
        }
    } catch (SQLException e) {
        throw new DaoException("No se pudo insertar la persona", e);
    }
}
```

IMPLEMENTACIÓN MYSQL: LISTAR



PersonaDAOMySQL — parte 3

```
@Override
public List<Persona> findAll() throws DaoException {
    List<Persona> personas = new ArrayList<>();
    String sql = "SELECT id, nombre, apellido, documento FROM personas";
    try (Connection cn = DriverManager.getConnection(url, usuario, clave);
         Statement st = cn.createStatement();
         ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            Persona p = new Persona(
                rs.getString("nombre"),
                rs.getString("apellido"),
                rs.getString("documento"));
            p.setId(rs.getLong("id"));
            personas.add(p);
        }
    } catch (SQLException e) {
        throw new DaoException("No se pudo listar las personas", e);
    }
    return personas;
}
```

VENTAJAS DEL PATRÓN DAO

DESACOPLAMIENTO

La lógica de negocio no depende de JDBC ni del motor de base de datos.

REUTILIZACIÓN

Las operaciones CRUD se escriben una sola vez y se usan desde toda la aplicación.

MANTENIBILIDAD

Un cambio de esquema o de motor se resuelve en un solo lugar del código.

TESTEABILIDAD

Se puede reemplazar la implementación real por una simulada para testear.



MENSAJE FINAL

"Separar el qué del cómo es la esencia del buen diseño."
El patrón DAO aplica esa idea al acceso a datos: la aplicación pide personas sin preguntar cómo se guardan.